

Procesamiento de Lenguaje Natural I

Vectorización de texto

Docentes:

MSc. Fernando Silva

MSc. Josselyn Ordoñez

emails: fernandosilva.clases2@gmail.com

josselynsofia@gmail.com

Programa del curso



Clase 1: Introducción a NLP, Vectorización de documentos.

Clase 2: Word embeddings.

Clase 3: Redes recurrentes: Elman, LSTM y GRU.

Clase 4: Modelos de lenguaje y generación de secuencias.

Clase 5: CNNs, introducción a atención. Modelos de clasificación.

Clase 6: Modelos Seq2seq.

Clase 7: Mecanismo de atención, Transformers.

Clase 8: Grandes modelos de lenguaje.

***Unidades con desafíos de código a presentar al finalizar el curso.**

(Clase 1) Una forma de representar palabras/oraciones es con BoW/BBow (TF,DF). Por ejemplo, "Hola, soy Cesar" se representa con un vector numerico. El problema es que estos vectores no capturan informacion semantica, y ademas ML no es tan bueno para datos no estructurados.

(Clase 2) Una forma de solventar esto es realizar proyecciones: Word embeddings que permite capturar la información semantica.

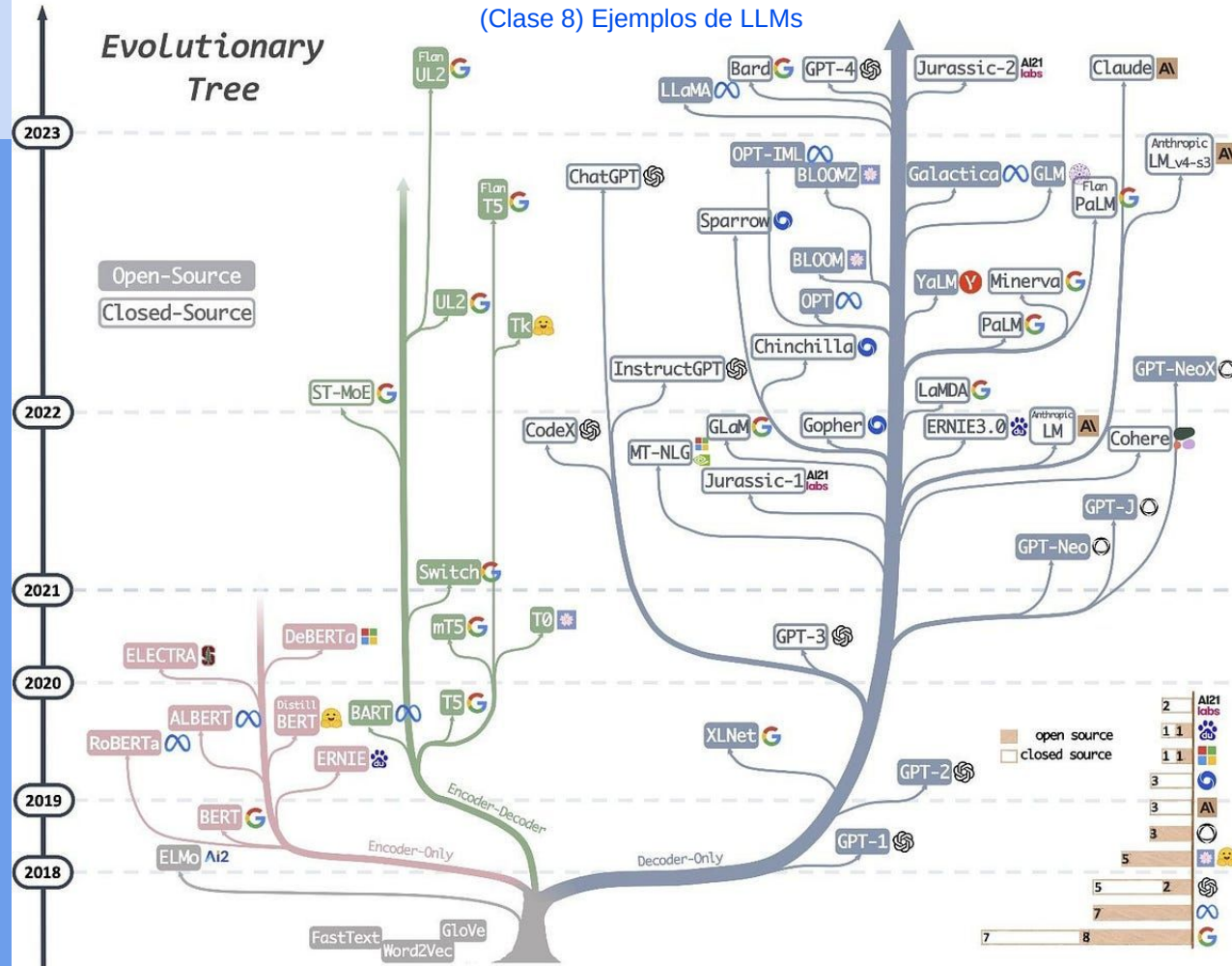
(Clase 3) Como los modelos de ML no son buenos para datos no estructurados, se utilizan redes especiales como las RNN.

(Clase 4-5-6) Las RNN tienen limitaciones, por ejemplo para generar texto se necesitan dos RNN (encoder-decoder) por lo cual el paso de informacion de una a otra es un cuello de botella. Solucion: "Cross Atencion" la cual se aplica en el modelo Seq2seq

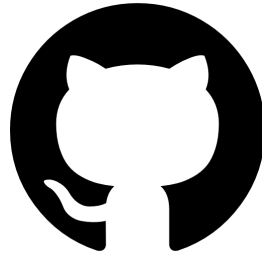
(Clase 7) Una limitacion importante de las RNN es que procesan la informacion de forma secuencial. Si se utiliza Atencion en la etapa de Representacion (encoder) como en la Generacion (decoder) se tiene un Transformer, con la ventaja de ser paralelizable y soporta entradas mas grandes.



(Clase 8) Ejemplos de LLMs

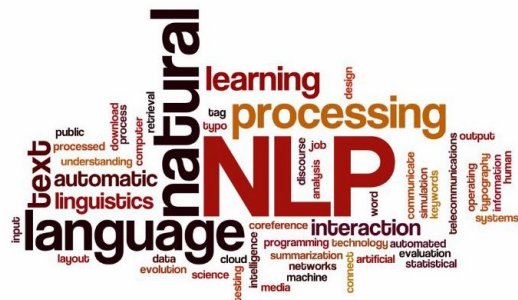


Link Github de la materia

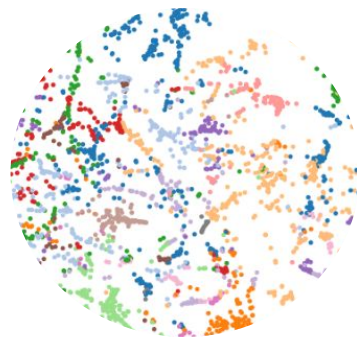


https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural

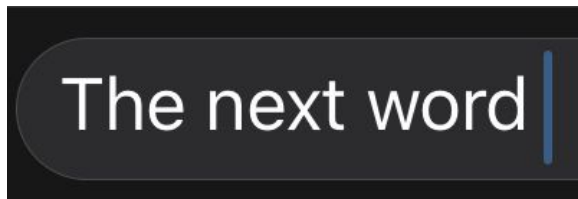
Desafíos propuestos



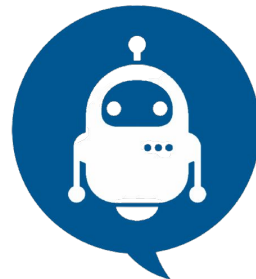
Vectorización de texto



Word embeddings



Generación de
secuencias

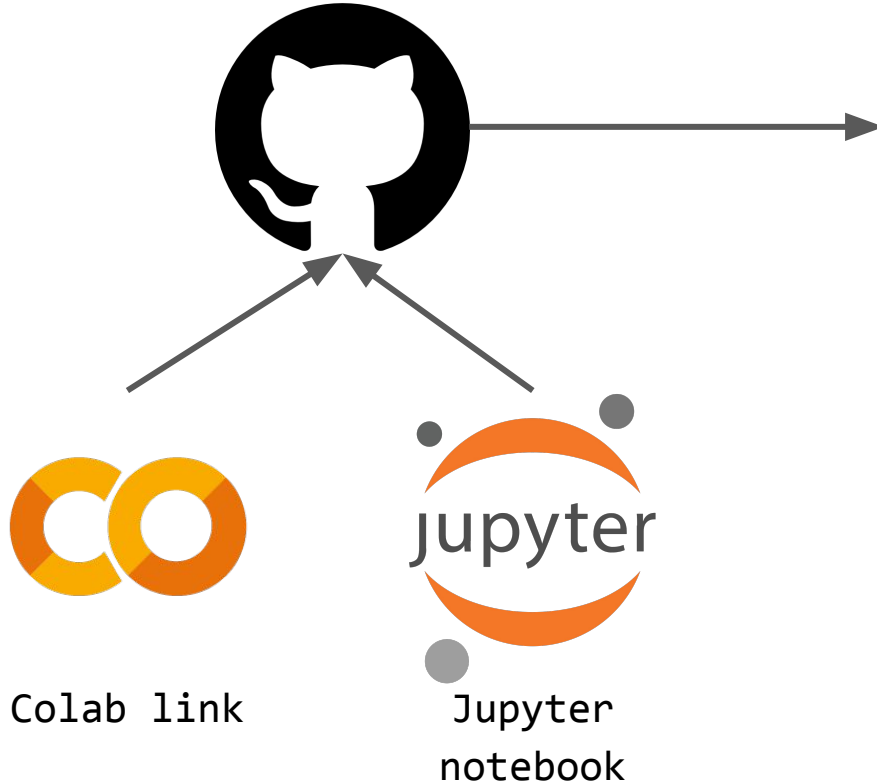


Modelos seq2seq

¿Cómo se entregan las resoluciones?



Su repositorio



Envían por mail el link del repositorio notificando que ya podemos observar su trabajo NºX

¿Cómo se evaluarán los desafíos?



Nota máxima si la primera entrega se hace hasta última hora de
Argentina del jueves de la clase...

cierre
de
notas

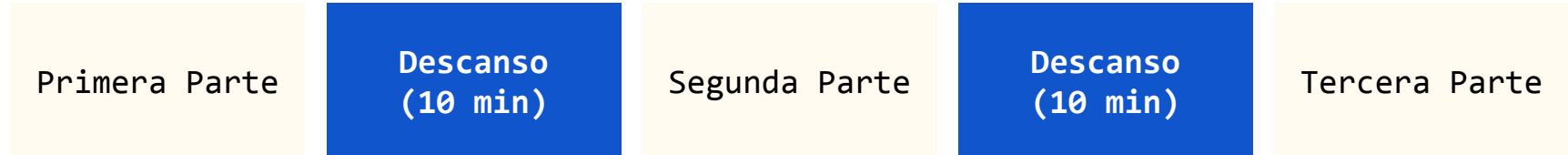
	1	2	3	4	5	6	7	8	
Desafío 1	10	10	10	9	9	8	7	6	5
Desafío 2		10	10	10	9	9	8	7	6
Desafío 3				10	10	10	9	8	7
Desafío 4						10	10	10	9

Los desafíos son individuales y deben ser subidos a un repositorio personal.

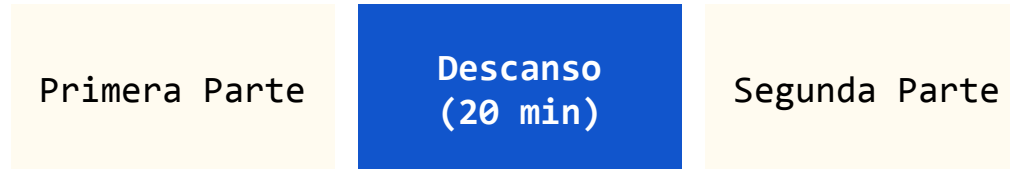
**PARA APROBAR EL CURSO TODOS LOS DESAFÍOS DEBEN SER ENTREGADOS Y EVALUADOS SATISFACTORIAMENTE
ANTES DE LA FECHA DEL CIERRE DE NOTAS**



Opción A:



Opción B:



Agenda



- Introducción al Procesamiento de Lenguaje Natural (NLP)
- Historia del NLP
- Aplicaciones de NLP
- Tareas de NLP
- Preprocesamiento de Texto
- Vectorización de Texto
- Machine Learning para NLP

Introducción al Procesamiento de Lenguaje Natural (NLP)

¿Qué es el lenguaje?



Desde un punto de vista de la Lingüística:

- El Lenguaje es un sistema de símbolos más reglas. Contempla semántica, sintaxis, y es pragmática.
- Chomsky menciona la idea de gramática generativa:
 - Donde tenemos un conjunto de reglas capaces de generar todas las oraciones correctas de un idioma.
 - Con pocas reglas podemos generar infinitas oraciones.
 - En NLP clásico se intentó modelar lenguaje con reglas tipo Chomsky.



Los NGRAMS representan una idea similar a las reglas lingüísticas.

¿Qué es el lenguaje?



Desde un punto de vista de la **Lingüística**:

- El Lenguaje es un sistema de símbolos más reglas. Contempla semántica, sintaxis, y es pragmática.
- ¿Los Large Language Models (LLMs) conocen gramática o solo patrones?
 - ¿son loros estocásticos? (**Clase 7 y 8**)

Los NGramas se utilizan en Language Models, por ejemplo Whatsapp predice la siguiente palabra.

On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜

Emily M. Bender*
ebender@uw.edu
University of Washington
Seattle, WA, USA

Angelina McMillan-Major
aymm@uw.edu
University of Washington
Seattle, WA, USA

Timnit Gebru*
timnit@blackinai.org
Black in AI
Palo Alto, CA, USA

Shmargaret Shmitchell
shmargaret.shmitchell@gmail.com
The Aether

On the Dangers of Stochastic Parrots:
Can Language Models Be Too Big? (2021)

¿Qué es el lenguaje?



Desde un punto de vista de la Filosofía:

- Ludwig Wittgenstein propone una idea clave:

“El significado de una palabra es su uso en el lenguaje.”

- Esto rompe con la idea clásica de que las palabras tienen significado fijo.
- Por ejemplo la palabra “banco” puede significar:
 - Banco financiero
 - Banco para sentarse
 - Banco de peces

El significado depende del contexto.



¿Qué es el lenguaje?



Desde un punto de vista de la **Filosofía**:

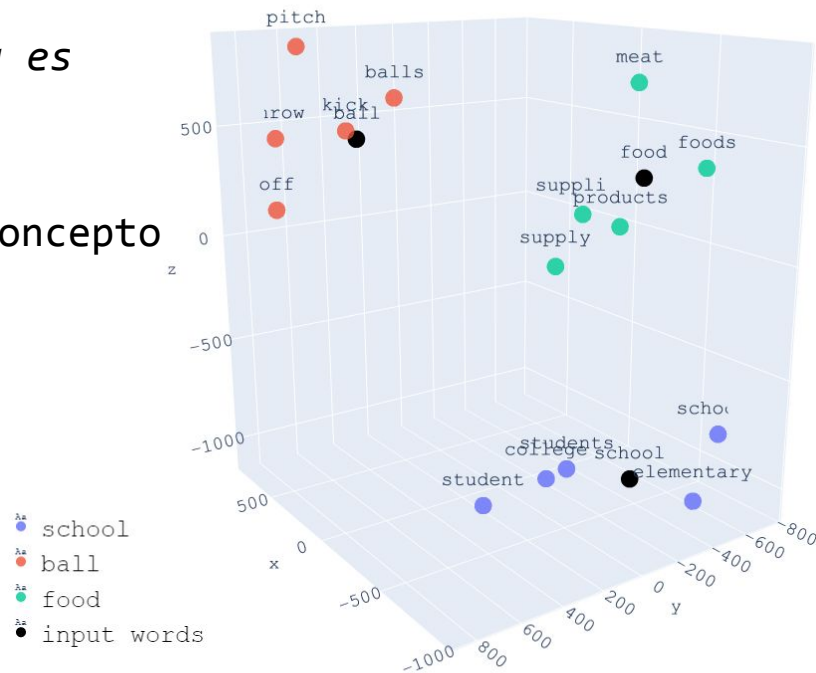
- Ludwig Wittgenstein propone una idea clave:

“El significado de una palabra es su uso en el lenguaje.”

- Esta idea está muy relacionada a un concepto llamado embeddings (**Clase 2**):

↳ Vector que captura relación semántica

En este grafico se redujo los embeddings a 3 dimensiones (ej. con TSNA o PCA).



¿Qué es el Procesamiento de Lenguaje Natural?



El procesamiento de lenguaje natural (PLN o NLP) es el área de la Inteligencia Artificial que desarrolla modelos computacionales capaces de representar, procesar, comprender y generar lenguaje humano mediante métodos algorítmicos y estadísticos.

Desde IA, NLP implica cuatro cosas clave:

- 1.- **Representación:** Transformar lenguaje (texto o voz) en estructuras matemáticas
- 2.- **Modelado:** Construir funciones que capturen regularidades del lenguaje.
- 3.- **Inferencia:** Dado un input lingüístico, producir una salida coherente.
- 4.- **Aprendizaje:** Ajustar parámetros a partir de datos textuales masivos.

Manifestaciones del Lenguaje



Lenguaje hablado
En NLP: Speech
Processing



Lenguaje escrito
En NLP: Desde N-Grams
hasta Transformers



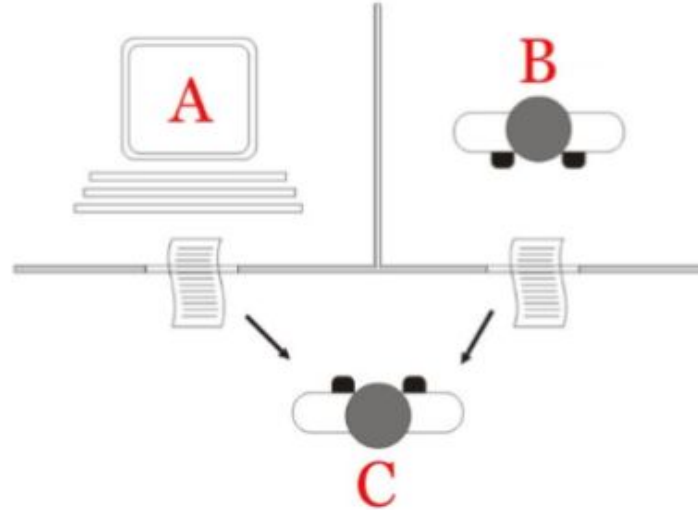
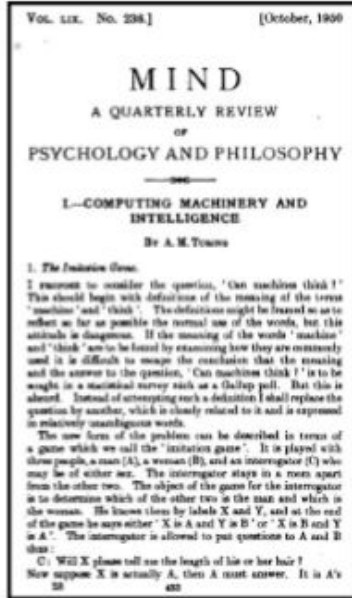
Lenguaje gestual
En NLP: Desde Sign
Language Recognition
hasta MultiModalidad



Lenguaje digital
En NLP: Desde Sentiment
analysis hasta Topic
modeling

Historia del NLP

Test de turing (1950)



¿Puede una máquina hablar como si fuera un humano?

Eliza (1964) - Primer bot NLP



Welcome to

```
EEEEEE LL      IIII ZZZZZZZZ  AAAAA
EE      LL      II       ZZ  AA  AA
EEEEEE LL      II       ZZZ  AAAAAA
EE      LL      II       ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZZZ AA  AA
```

Eliza is a mock Rogerian psychotherapist.

The original program was described by Joseph Weizenbaum in 1966.

This implementation by Norbert Landsteiner 2005.

ELIZA: Please tell me what's been bothering you.

YOU: I don't really know

ELIZA: Do you wish to be able to really know ?

YOU: Hmm.. well, I guess yes

ELIZA: Do you say you guess yes for some special reason ?

YOU: I don't really know

ELIZA: Does that trouble you ?

YOU: What do you think?

ELIZA: What are your feelings now ?

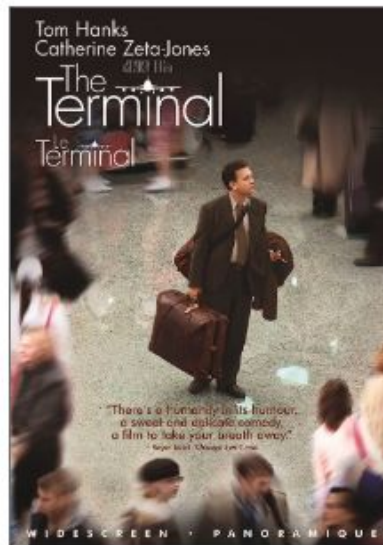
YOU: Sad

ELIZA: Does talking about this bother you ?

YOU: █

Se basaba en encontrar
las palabras claves del
texto y utilizarlas para
formular una pregunta

Statistical Machine Translation (1990)



IBM introduce los SMT. Los cuales se basan en probabilidad para traducir texto de un idioma a otro.

Estos modelos son conocidos por su enfoque de alineación de palabras y su uso de datos paralelos para aprender traducciones.

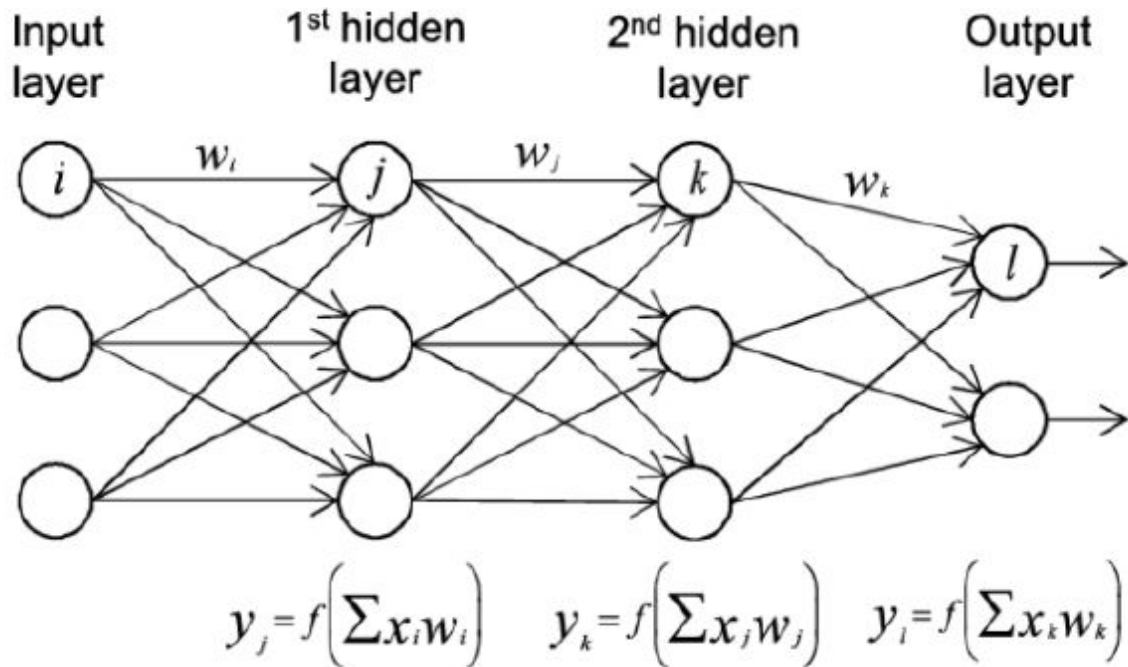
Se puede implementar un modelo de traducción aprendiendo de a pares.

Dataset: <https://www.manythings.org/anki/>

Deep Learning (2012 - ...)



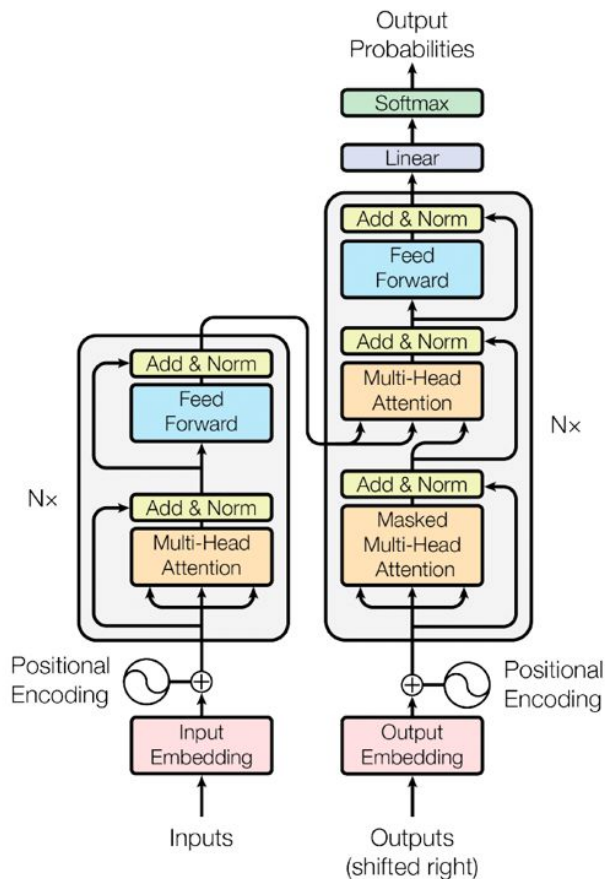
Deep Learning se hizo muy popular con AlexNet



Reduce esfuerzos
manuales.

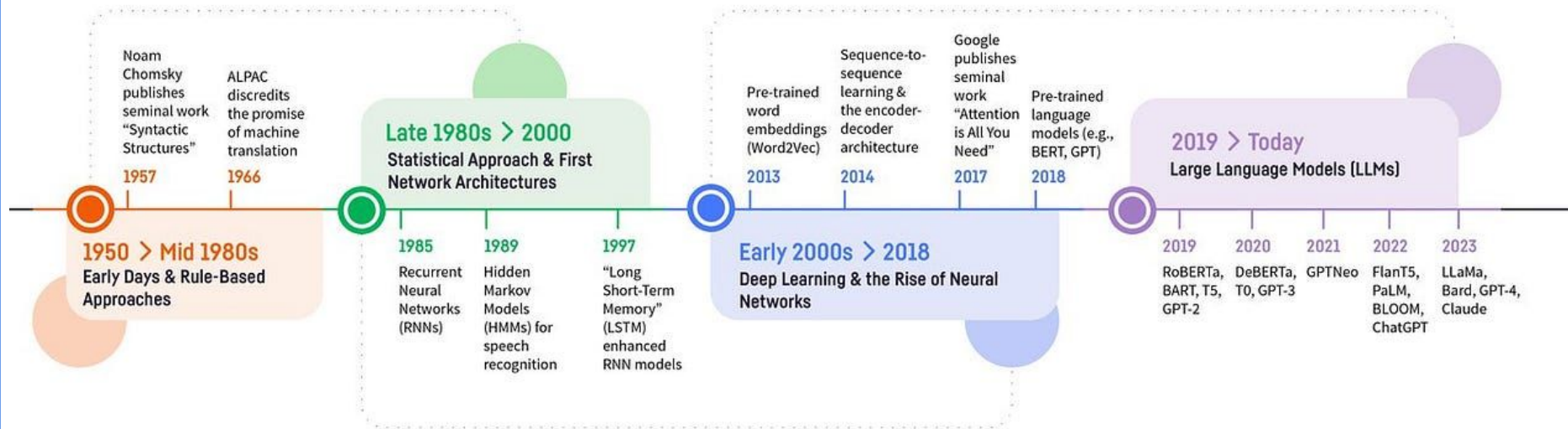
Tres ingredientes
esenciales: Hardware,
Data e Investigación

Transformer y Generative AI (2017 - ...)



En 2017 se presenta el paper “Attention is All You Need” donde se propone la arquitectura Transformer, marcando así el inicio de Generative AI.





Créditos: Dataiku

Aplicaciones de NLP

Machine Translation



how do I say "hello world" in french



All

Images

Shopping

Videos

News

More

Settings

Tools

About 2,660,000 results (0.71 seconds)

English - detected ▼



French ▼

hello world



Bonjour le monde



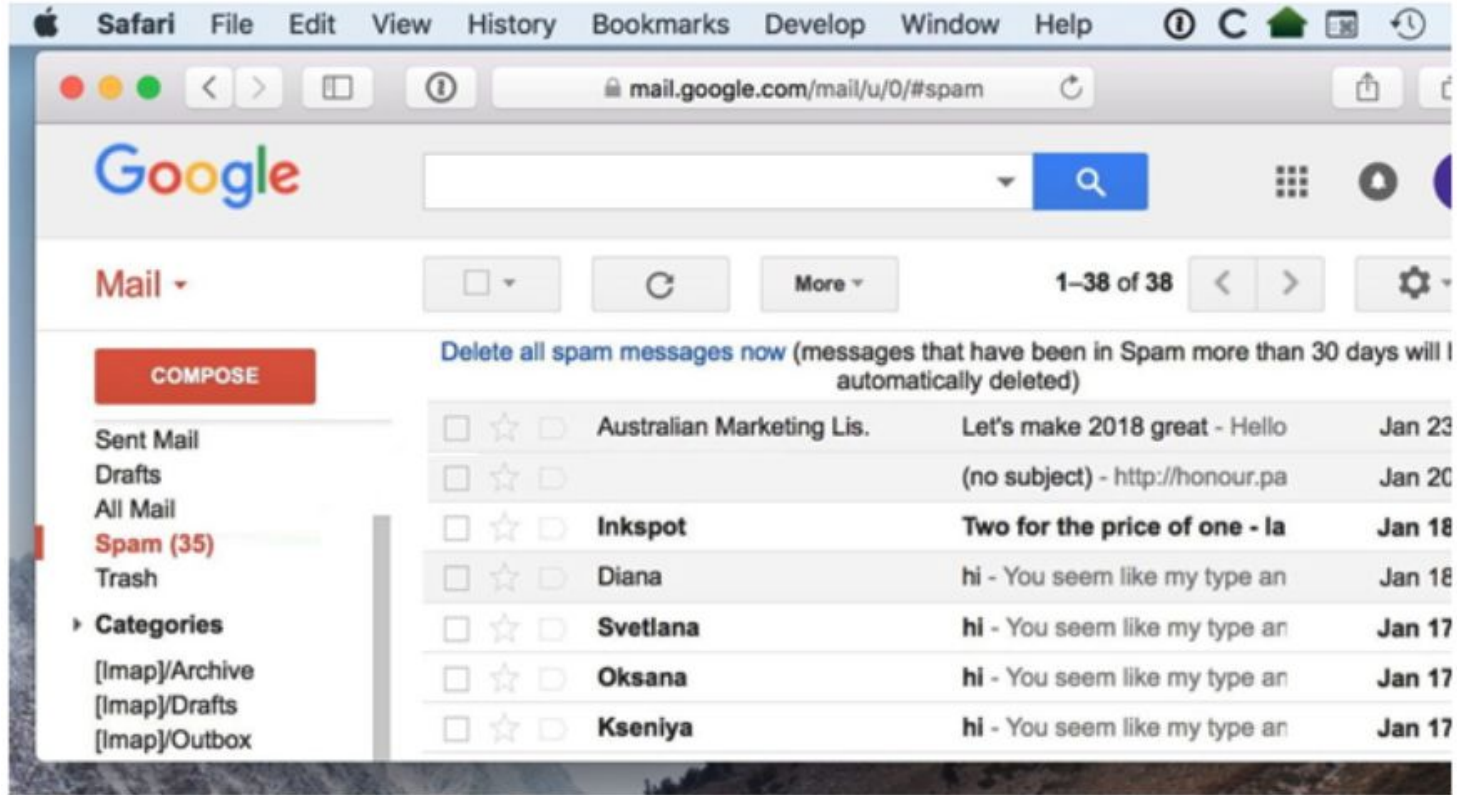
[Open in Google Translate](#)

[Feedback](#)

Asistentes de Voz



Clasificación de Texto



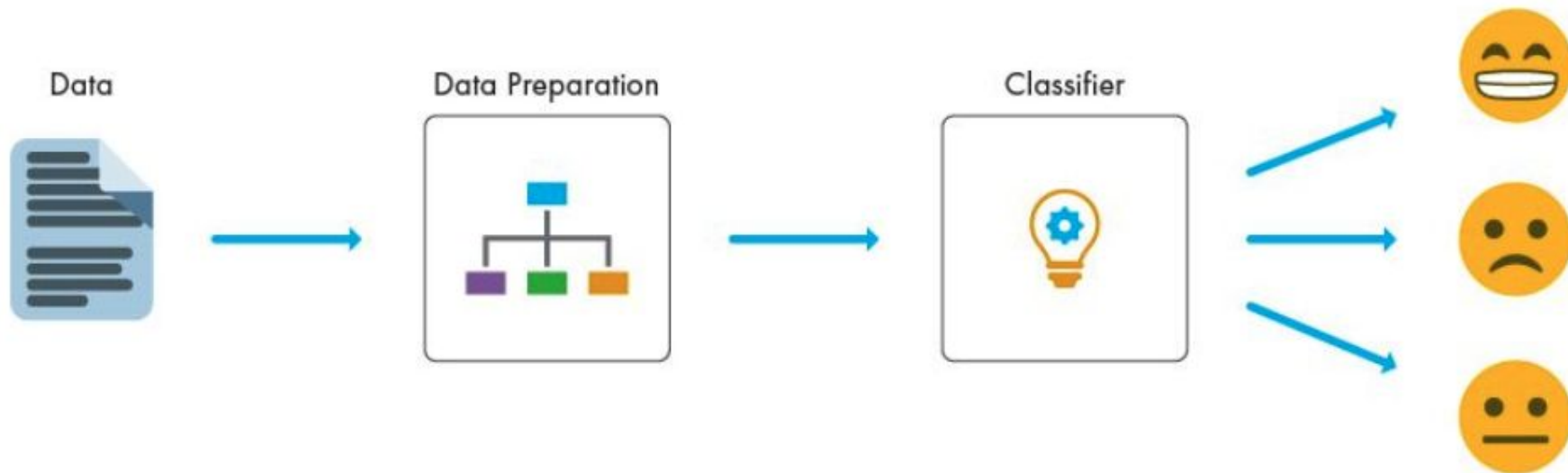
Chatbot

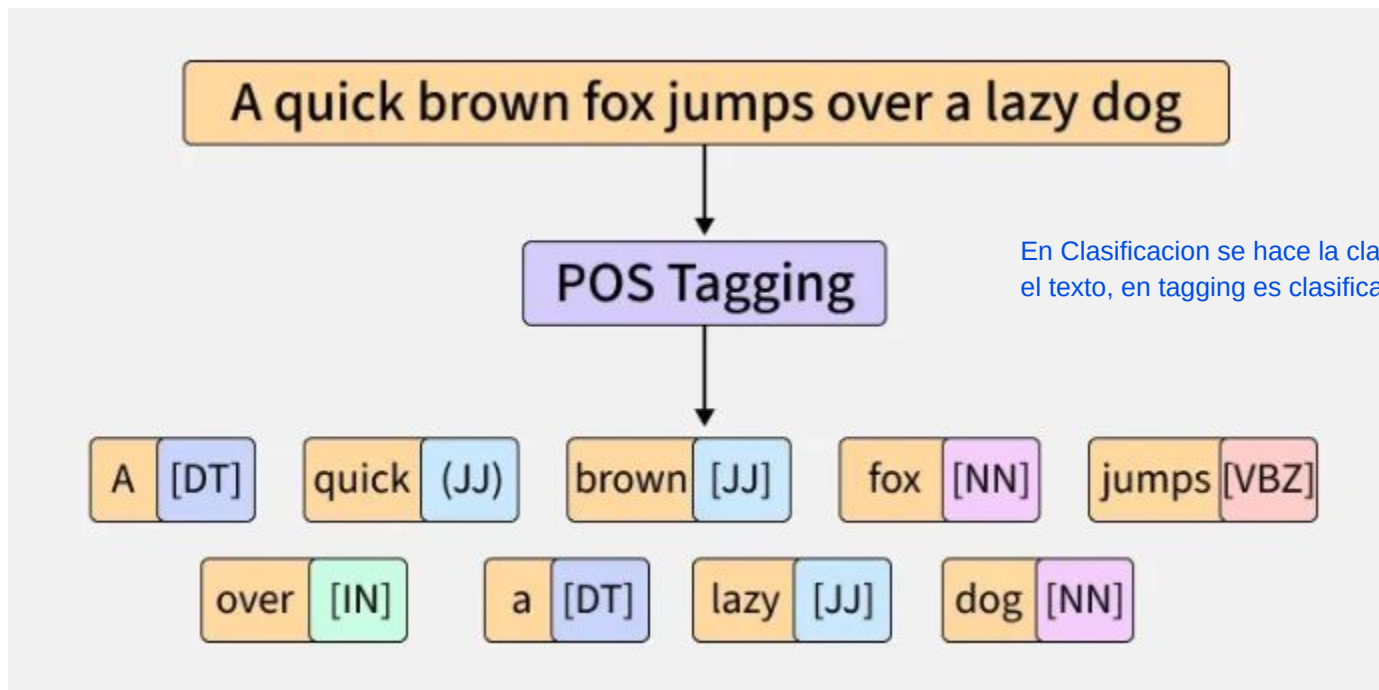


Tareas de NLP

(están asociadas a las aplicaciones de NLP)

Clasificación





En Clasificación se hace la clasificación de todo el texto, en tagging es clasificación granular

Tagging



Person

p

Loc

l

Org

o

Event

e

Date

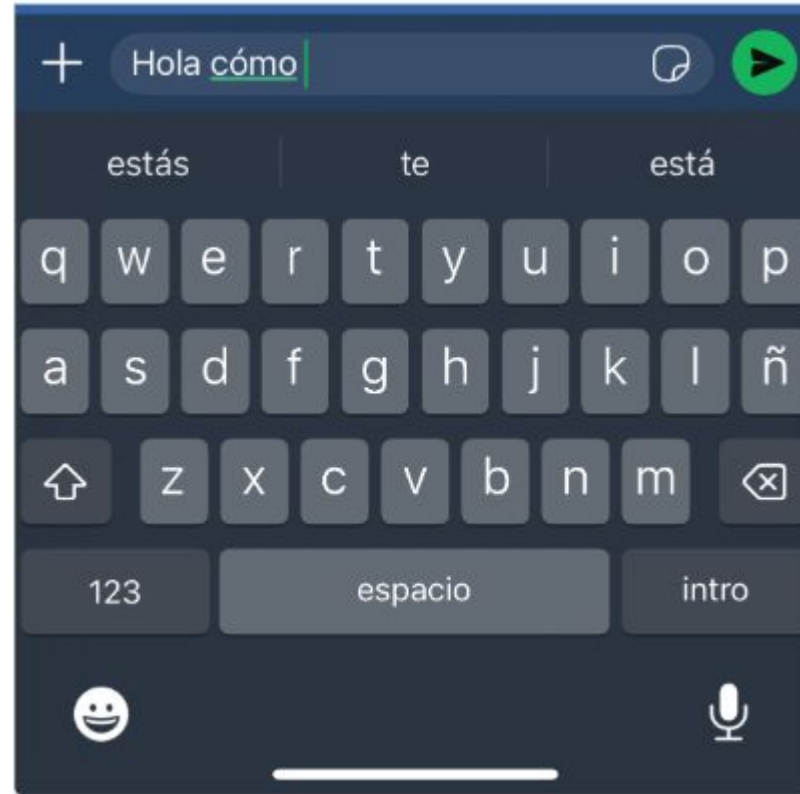
d

Other

z

Barack Hussein Obama II ✕ (born August 4, 1961 ✕) is an American ✕ attorney and politician who served as the 44th President of the United States ✕ from January 20, 2009 ✕, to January 20, 2017 ✕. A member of the Democratic Party ✕, he was the first African American ✕ to serve as president. He was previously a United States Senator ✕ from Illinois ✕ and a member of the Illinois State Senate ✕.

Autocompletado



El autocompletado utiliza NGrams o Cadenas de Markov. Es la base de generación de texto de los LLMs

Generación de texto



ChatGPT ▾

 Oferta gratis

 Memoria completa

¿Por dónde empezamos?

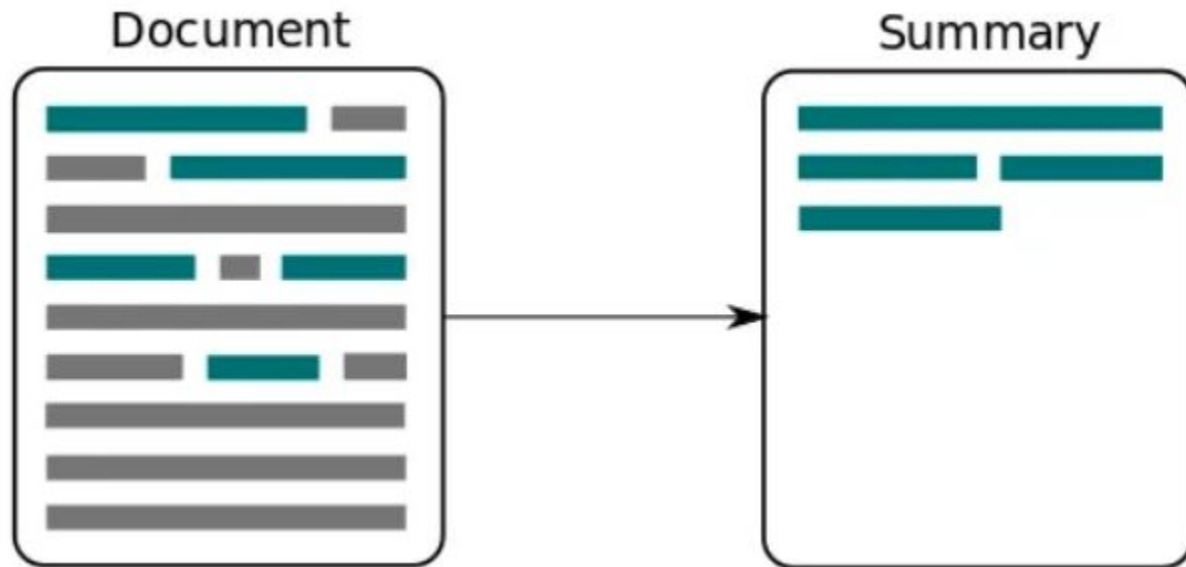
+ Pregunta lo que quieras



Summarization

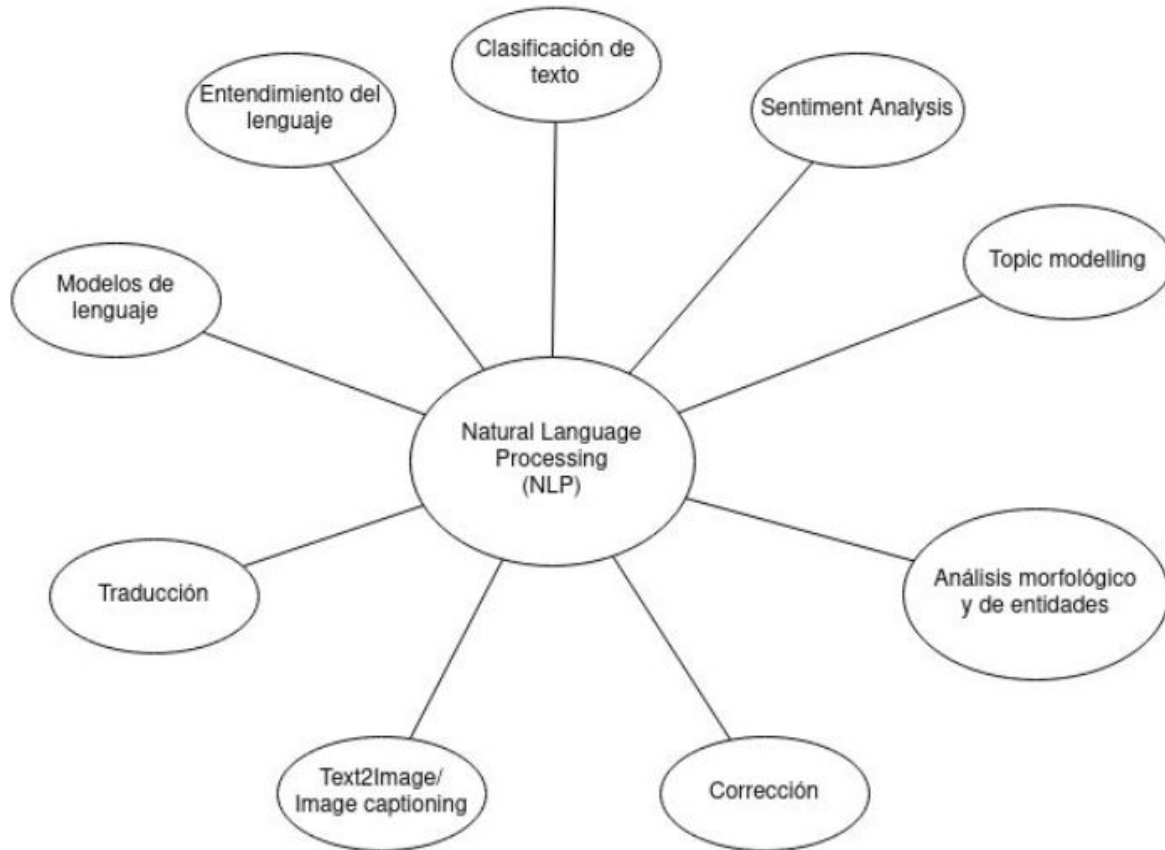


La generacion de resumen funciona muy bien con LLMs,
por ejemplo con T5.





Tareas de NLP

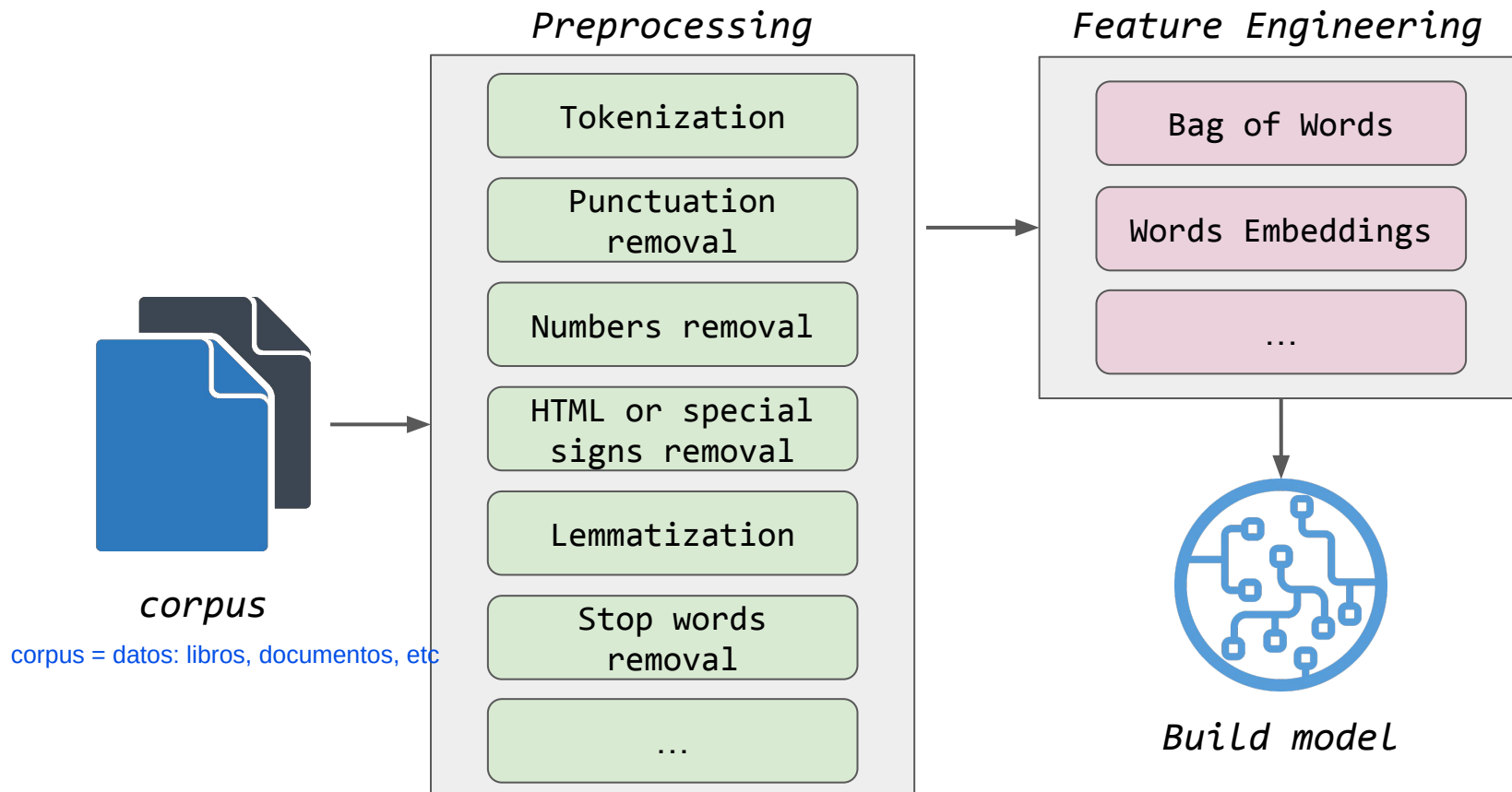


Preprocesamiento de texto (en NLP clásico)

Hoy en día este procesamiento no se hace en los LLMs pero ciertos conceptos se mantienen como la tokenización.

Preprocesamiento de texto

[LINK GLOSARIO](#)



Normalización básica



- Lowercasing
- Eliminar signos de puntuación
- Quitar caracteres raros

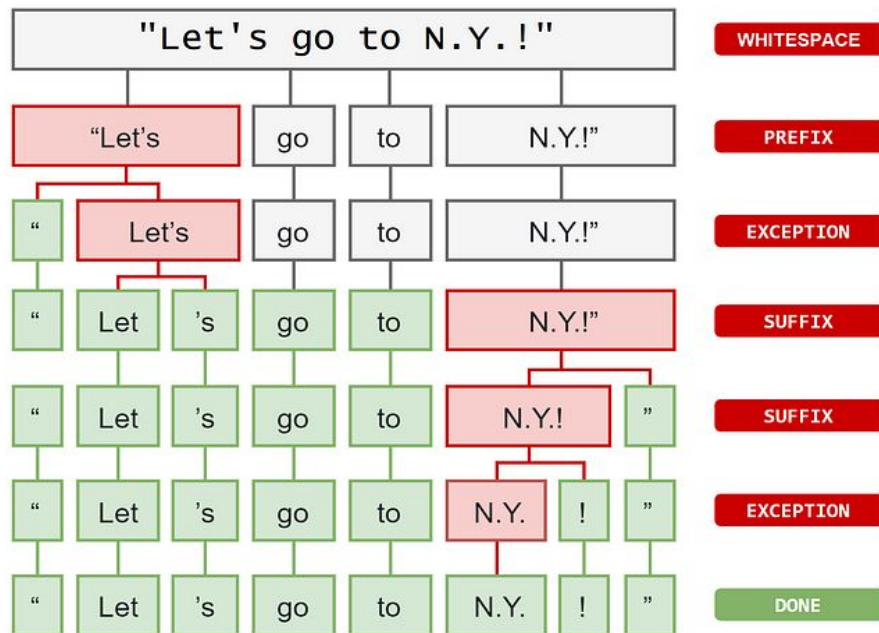


Tokenization



Proceso en el cual una oración o documento es segmentado en términos individuales. Una vez finalizada la segmentación cada término único es referenciado mediante un token.

Ejemplo de tokenización: dividir las oraciones en palabras, aunque sería mas eficiente dividir en "silabas"



Tokenization

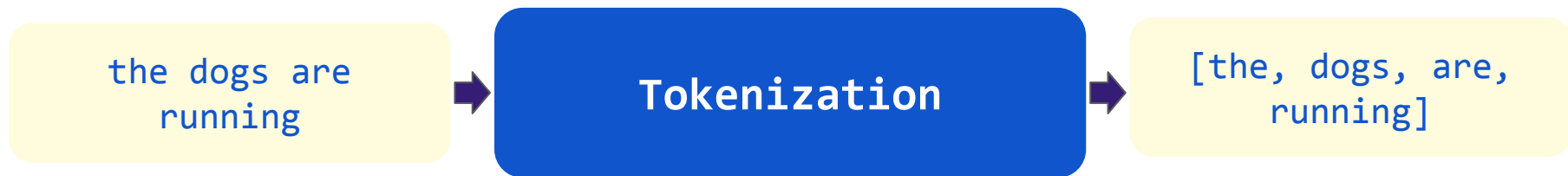


A primera vista parece que tokenizar es “solo hacer split”, pero tenemos los siguientes escenarios:

Como estos escenarios son un problema, se utilizan librerías que ya tienen en cuenta estas cosas. Por ejemplo con NLTK

- abreviaturas como **D.C.**
- nombres compuestos como **New York**
- contracciones como “**doesn't**” ó “**could't've**”
- nombres como “**SARS-Cov-2**” o “**R2-D2**”
- con idiomas como el **chino**, que no tienen espacios en blanco
- o idiomas como el **turco**, donde una sola palabra puede expresar tanta información como una oración completa

Tokenization



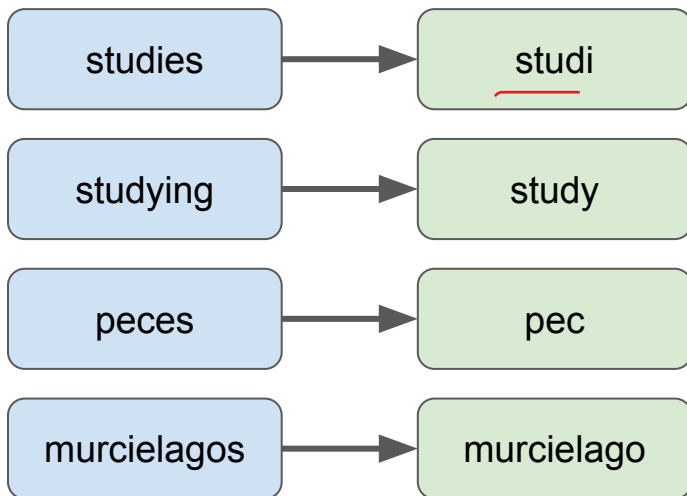
Un problema de la tokenizacion es que al formar el vocabulario, el modelo solo sabe responder con dicho vocabulario. Por lo cual es conveniente tokenizar por "silabas" mas que en palabras (con las silabas aumenta el vocabulario)

Stemming



Aplica reglas de eliminación de patrones recurrentes de la lengua.
El resultado es una palabra truncada que no será necesariamente la raíz morfológica de la palabra.

Regla: Eliminar los sufijos



plural irregular

especimen, especímenes

régimen, regímenes

Stemming



La ventaja de Stemming es que es rapido y permite reducir el vocabulario. Pero no siempre acierta en la palabra truncada

[the, dogs, are,
running]



Stemming

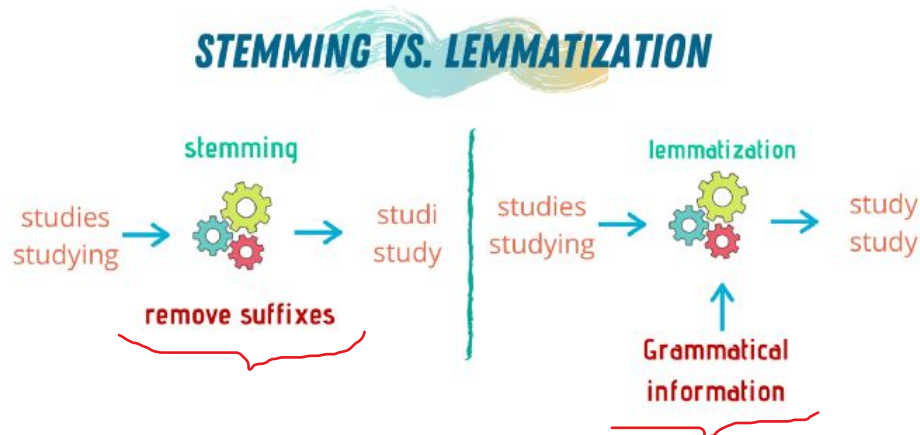
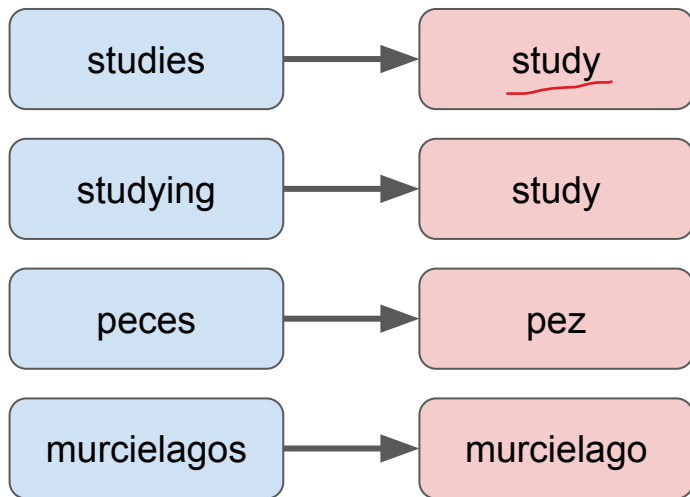


[the, dog, are,
runn]

Lematización (lemmatization)



Devuelve la raíz morfológica de una palabra. Para ello se necesita un diccionario del idioma con todas las declinaciones posibles de las palabras raíz.



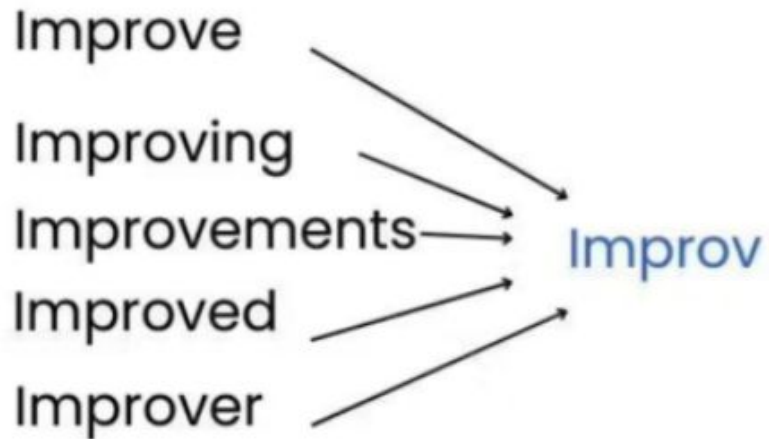
Lematización (lemmatization)



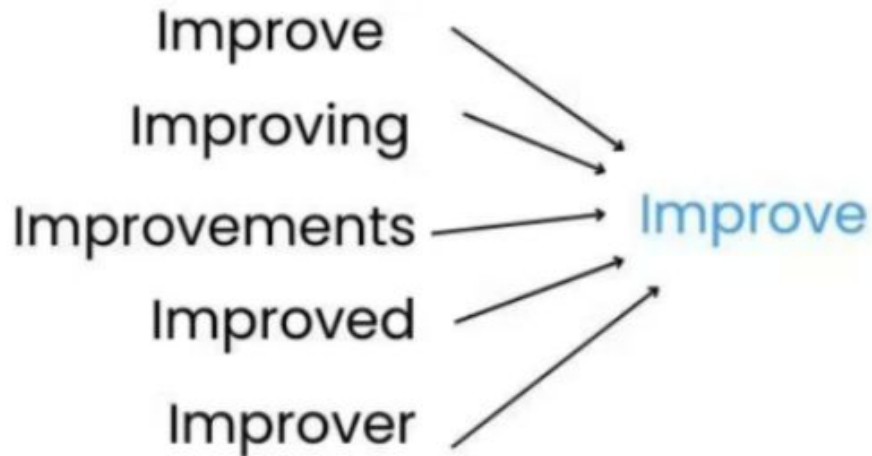
La lematización encuentra la palabra raíz pero es más lento que Stemming



Stemming vs Lemmatization



Stemming



Lemmatization

Stemming vs Lemmatization



Stemming

Ventajas: Rápido y simple.

Desventajas: Puede producir raíces no válidas y perder precisión semántica.

Lemmatization

Ventajas: Produce palabras válidas y mantiene la precisión semántica.

Desventajas: Puede ser más lenta y requiere recursos adicionales (como WordNet).

↪ Provee las reglas lingüísticas incluso de varios idiomas.

Practiquemos lo visto hasta ahora



Link al Colab



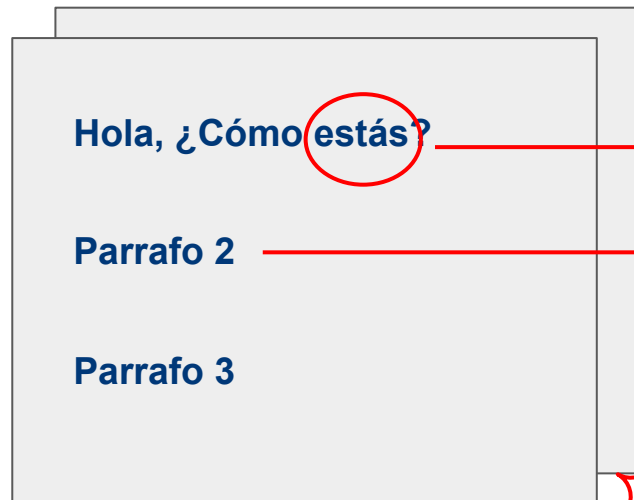
LINK

Vectorización de Texto

Vectorización de texto



[LINK GLOSARIO](#)



Término t : palabra/símbolo "t" del documento

Document: su largo es variable, normalmente una sentencia/oración/párrafo.

Corpus: conjunto de documentos, forman todo el vocabulario.

No podemos ingresar texto
a una red
¿Cómo transformamos
palabras a números?

vectorización

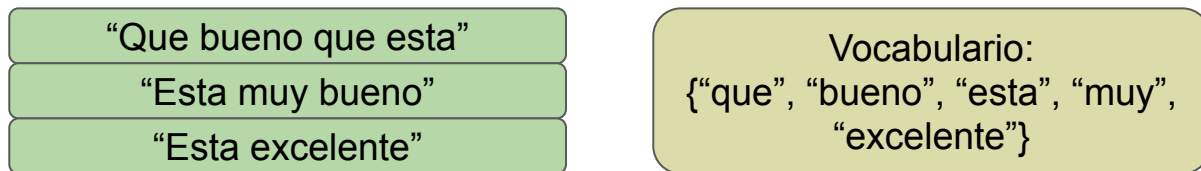


Vectores de
palabras/documentos

Vectores de frecuencia/conteo



"Por cada documento en el corpus se calcula un vector que representa cuántas veces cada palabra del vocabulario aparece en ese documento"



CountVectorizer

Por lo cual puede haber muchos ceros, esto se conoce como "sparse". Lo cual es un problema

que	bueno	esta	muy	excelente
2	1	1	0	0
0	1	1	1	0
0	0	1	0	1

Los vectores tienen el tamaño del vocabulario

Bolsa de palabras (BOW)



Raw Text

Bag-of-words
vector

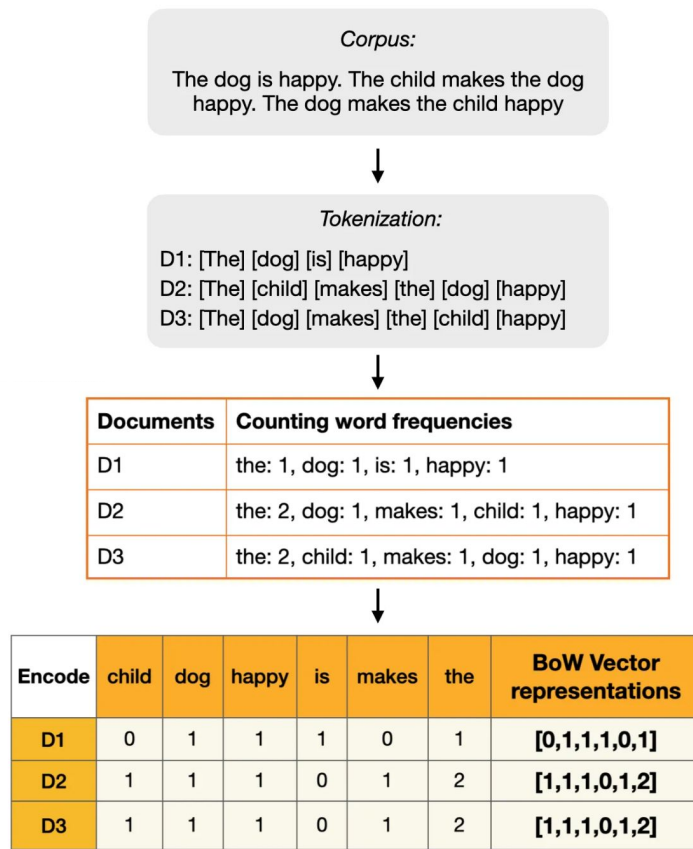
it is a puppy and it
is extremely cute



it	2
they	0
puppy	1
and	1
cat	0
aardvark	0
cute	1
extremely	1
...	...

Bag of Words representa un documento mediante un vector basado en la presencia o frecuencia de palabras del vocabulario, ignorando el orden en el que aparecen.

Bolsa de palabras (BOW)



Bag of Words representa un documento mediante un vector basado en la presencia o frecuencia de palabras del vocabulario, **ignorando el orden en el que aparecen.**

Mismo vector! BoW sufre de sparse y ademas no captura la relacion semantica

Bolsa de palabras (BOW)



Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



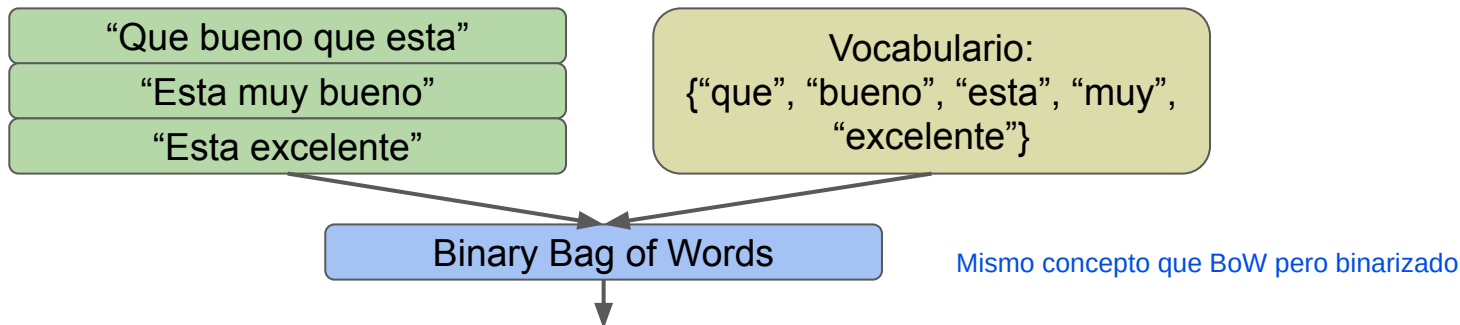
	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Bag of Words representa un documento mediante un vector basado en la presencia o frecuencia de palabras del vocabulario, **ignorando el orden en el que aparecen.**

Vectores Binary Bag of Words (BBoW)



"Por cada documento en el corpus se calcula un vector que representa si cada palabra del vocabulario aparece o no en ese documento"



que	bueno	esta	muy	excelente
1	1	1	0	0
0	1	1	1	0
0	0	1	0	1

Stop words



Palabras que no aportan valor al significado de una oración ya que son muy frecuentes o comunes en el lenguaje

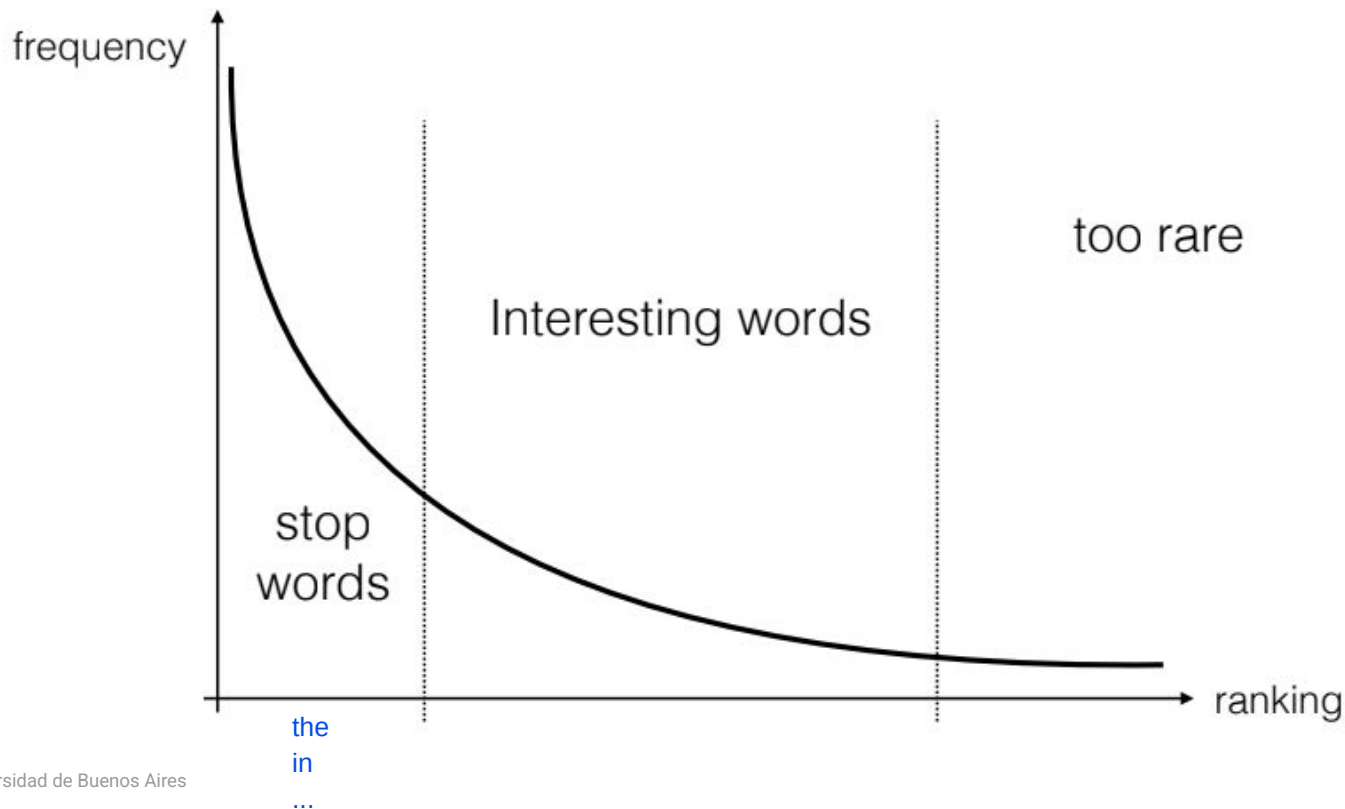


Argentina, oficialmente, República Argentina, es un país soberano **de** América **del** Sur, ubicado **en** el extremo sur y sudeste **de** dicho subcontinente. Adopta **la** forma **de** gobierno republicana, democrática, representativa y federal.

Argentina, oficialmente, República Argentina, es país soberano América Sur, ubicado extremo sur sudeste dicho subcontinente. Adopta forma gobierno republicana, democrática, representativa federal.

Stop words

Zipf's Law



TF-IDF (Term frequency-Inverse document frequency)



Se utiliza como indicador de cuán importante es una palabra (término) en un documento.

$$\text{TF-IDF}_{(n,d)} = \text{TF}_{(n,d)} \times \text{IDF}_{(n)}$$

Peso de un término (n) en un documento (d)

Frecuencia de aparición de un término (n) en un documento (d)

Factor IDF de un término (n)

En TF-IDF se penalizan (asignando un peso bajo) las palabras que son muy frecuentes para no tenerlas en cuenta

Vector IDF (Inverse Document Frequency)



"Proporción de documentos en el corpus que poseen el término"

También suele utilizarse el logaritmo en base 2, su función es conseguir un coeficiente bajo, fácil de manejar

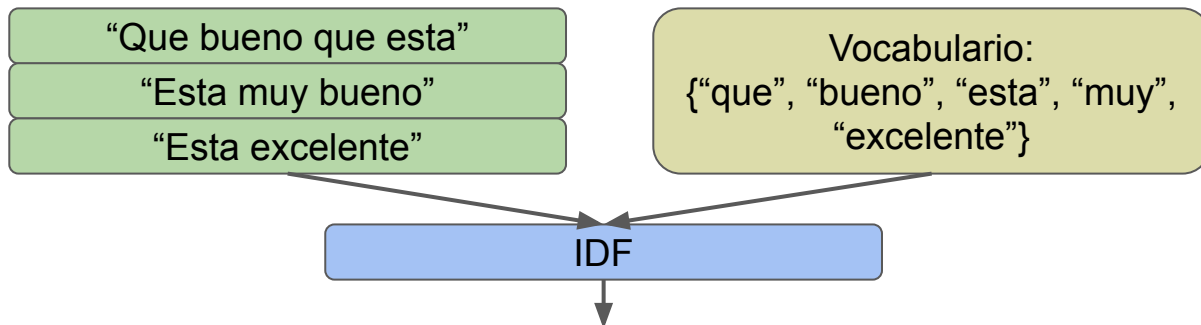
$$\text{IDF}_{(n)} = \log_{10} \frac{N}{\text{DF}_{(n)}}$$

N es el número total de documentos de la colección.

DF (Document Frequency) es el número documentos en los que aparece el término (n) a lo largo de toda la colección

Si el término aparece en todos los documentos el IDF será cero (es popular y por lo tanto aporta poco valor)

Vector IDF



que	bueno	esta	muy	excelente
$\log(3/1)$	$\log(3/2)$	$\log(3/3)$	$\log(3/1)$	$\log(3/1)$
0.477	0.176	0	0.477	0.477

que	bueno	esta	muy	excelente
0.477	0.176	0	0.477	0.477

Se obtiene como la división de la cantidad de documentos sobre la suma en axis=0 (vertical) del CountVectorizer.

Vector TF-IDF



“Que bueno que esta”

“Esta muy bueno”

“Esta excelente”

Vocabulario:
{“que”, “bueno”, “esta”, “muy”,
“excelente”}

IDF

que	bueno	esta	muy	excelente
$\log(3/1)$	$\log(3/2)$	$\log(3/3)$	$\log(3/1)$	$\log(3/1)$

TF-IDF

que	bueno	esta	muy	excelente
$2 * \log(3/1)$	$1 * \log(3/2)$	$1 * \log(3/3)$	$0 * \log(3/1)$	$0 * \log(3/1)$
$0 * \log(3/1)$	$1 * \log(3/2)$	$1 * \log(3/3)$	$1 * \log(3/1)$	$0 * \log(3/1)$
$0 * \log(3/1)$	$0 * \log(3/2)$	$1 * \log(3/3)$	$0 * \log(3/1)$	$1 * \log(3/1)$

Similitud entre documentos



¿Cómo medimos qué tan parecidos son dos textos o palabras comparando el significado de sus vectores?

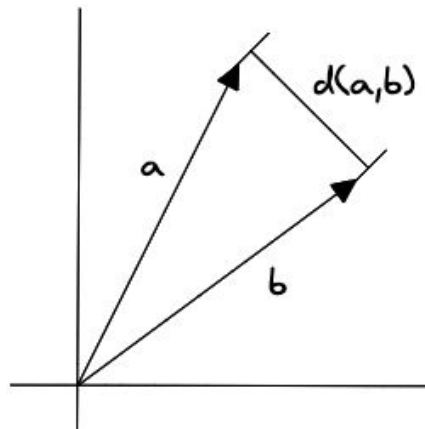
En NLP clásico (TF-IDF, BoW):

- Palabras frecuentes pueden tener vectores con mayor norma.

Si usamos Euclidiana:

- Documentos largos parecen más lejos
- Palabras frecuentes dominan

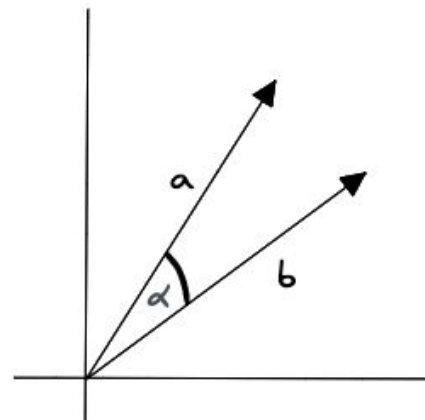
Coseno normaliza eso.



Distancia Euclidean

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

Mide distancia absoluta en el espacio.
Depende fuertemente de la magnitud.



Similitud Coseno

(muy utilizado en embedding)

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Mide el ángulo entre vectores.
Ignora la magnitud y compara orientación.

Similitud coseno



"Se utiliza para evaluar la dirección de dos vectores"

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Similitud coseno = 1 → los vectores tienen la misma dirección.

Similitud coseno = 0 → los vectores son ortogonales.

Similitud coseno = -1 → los vectores apuntan en sentido contrario.

Intuición de la similitud coseno



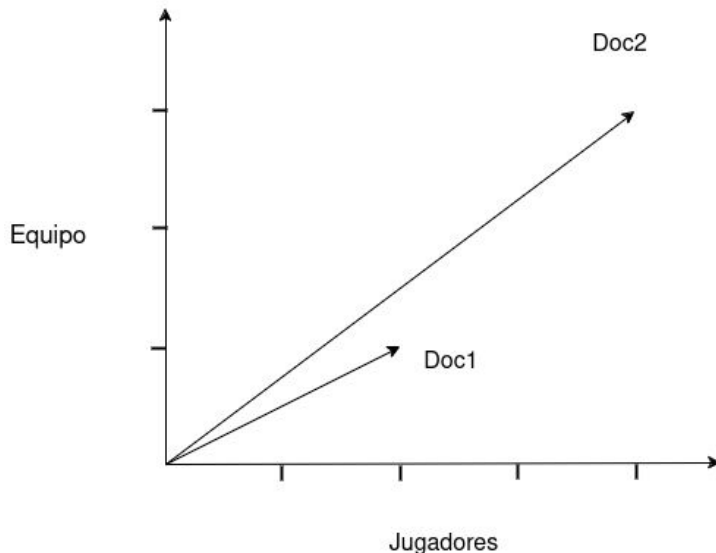
Doc1

"Cada equipo en el campo tiene hasta once jugadores..."



Doc2

"... el equipo Argentino presentó a todos sus jugadores titulares..."



Para la distancia euclídea, los documentos son muy distintos. Para la similitud coseno, son similares.

Co-Occurrence Matrix



Una co-occurrence matrix (matriz de co-ocurrencia) es una representación que cuenta cuántas veces dos palabras aparecen juntas dentro de un mismo contexto (por ejemplo, una ventana de palabras).

A diferencia de Bag of Words, que representa documentos, la matriz de co-ocurrencia representa relaciones entre palabras.

context words:
4 words to the left +
4 words to the right

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

¡También nos da representaciones sparse!

Para solucionar la sparsidad, normalmente se aplican técnicas como:

- Reducción de dimensionalidad (SVD)
- Factorización de matrices (idea de GloVe)

Esparsidad de los vectores de conteos



One-Hot Encoding

The quick brown fox jumped over the brown dog



	cat	the	quick	brown	fox	jumped	over	dog	bird	flew	...	kangaroo	house
time	0	1	0	0	0	0	0	0	0	0	...	0	0
	0	0	1	0	0	0	0	0	0	0	...	0	0
	0	0	0	1	0	0	0	0	0	0	...	0	0
	0	0	0	0	1	0	0	0	0	0	...	0	0
	0	0	0	0	0	1	0	0	0	0	...	0	0
	0	0	0	0	0	0	1	0	0	0	...	0	0
	0	1	0	0	0	0	0	0	0	0	...	0	0
	0	0	0	1	0	0	0	0	0	0	...	0	0
	0	0	0	0	0	0	0	1	0	0	...	0	0
											← Dictionary Size →		

¡El idioma inglés tiene más de 180.000 palabras en su vocabulario en uso!

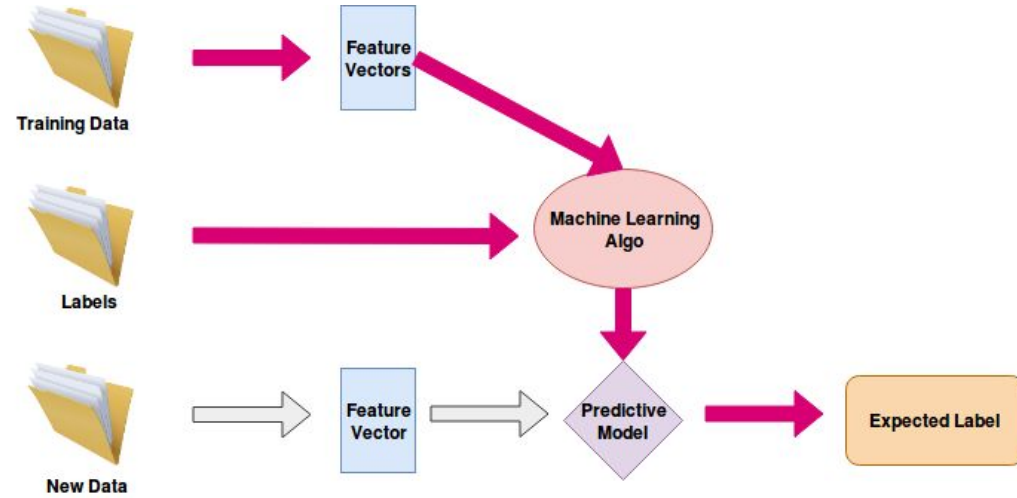
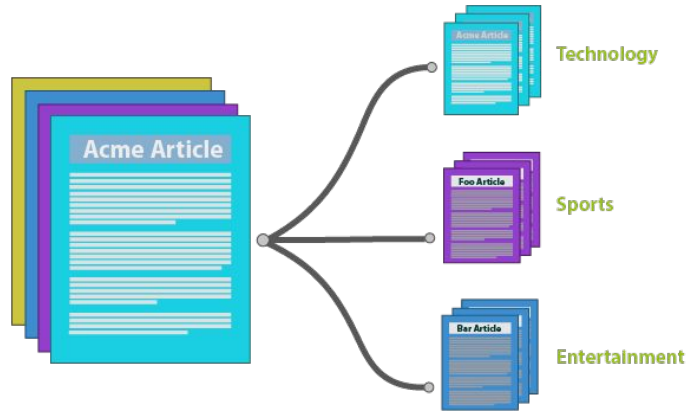
¡La representación es sumamente mala!
(poco densa)

No estamos aprovechando eficientemente la dimensionalidad del espacio de vectores. Hay muchos ceros

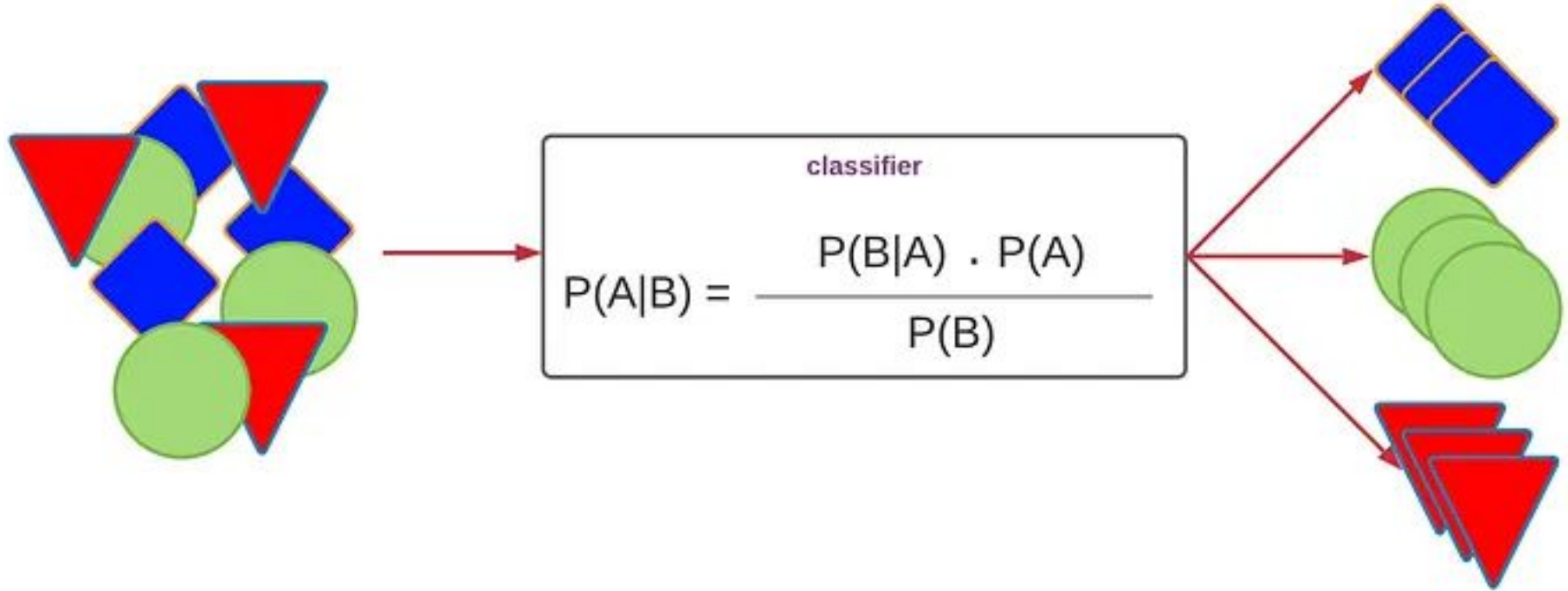
Solucion: usar redes neuronales para obtener representaciones que capturen la representacion semantica

Machine Learning para NLP

Modelo de clasificación de texto



Modelo de clasificación, Naïve Bayes



Modelo de clasificación, Naïve Bayes



Usaremos Naïve Bayes (Multinomial).

Queremos clasificar una frase como: { Positivo
Negativo

Los pasos que seguiremos serán:

- Paso 1: Obtenemos dataset de entrenamiento
- Paso 2: Construimos el vocabulario
- Paso 3: Calculamos probabilidades a priori
- Paso 4: Realizamos conteo de palabras por clase
- Paso 5: Aplicamos Laplace Smoothing Evita el problema de una palabra con conteo cero
- Paso 6: Clasificamos una nueva frase

Modelo de clasificación, Naïve Bayes



Paso 1: Obtenemos dataset de entrenamiento

Clase Positivo (2 documentos)

- "me gustó la película"
- "excelente película"

Clase Negativo (2 documentos)

- "mala película"
- "Película horrible"



Paso 2: Construimos el vocabulario

"me gustó la
película", "excelente
película", "mala
película", "Película
horrible"



**Normalización de
Texto + Tokenización**

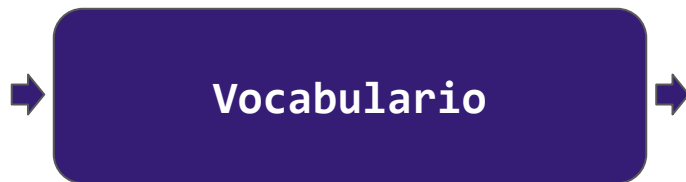


[me,gustó,la,película],
[excelente,película][exce
lente, película],
[excelente,película],
[mala,película][mala,
película][mala,película],
[película,horrible][pelíc
ula,horrible][película,ho
rrible]



Paso 2: Construimos el vocabulario

```
[me,gustó,la,película],  
[excelente,película][exce  
lente, película],  
[excelente,película],  
[mala,película][mala,  
película][mala,película],  
[película,horrible][pelíc  
ula,horrible][película,ho  
rrible]
```



$V = \{\text{me, gustó, la, excelente, mala, horrible, película}\}$

$$|V| = 7$$



Paso 3: Calculamos probabilidades a priori

Como hay 2 positivos y 2 negativos:

$$P(Positivo) = \frac{2}{4} = 0.5$$

$$P(Negativo) = \frac{2}{4} = 0.5$$

Clase Positivo (2 documentos)

- "me gustó la película"
- "excelente película"

Clase Negativo (2 documentos)

- "mala película"
- "Película horrible"



Paso 4: Realizamos conteo de palabras por clase

Total palabras en clase Positivo:

$$N_{pos} = 6$$

Total palabras en clase Negativo:

$$N_{neg} = 4$$

Clase Positivo (2 documentos)

- "me gustó la película"
- "excelente película"

Clase Negativo (2 documentos)

- "mala película"
- "Película horrible"



Paso 5: Laplace Smoothing

Fórmula general: $P(w|c) = \frac{\text{count}(w, c) + 1}{N_c + |V|}$

Probabilidades en Positivo

$$N_{pos} + |V| = 6 + 7 = 13$$

Ejemplos:

$$P(\text{excelente}|Pos) = \frac{1 + 1}{13} = \frac{2}{13}$$

$$P(\text{película}|Pos) = \frac{2 + 1}{13} = \frac{3}{13}$$

$$P(\text{mala}|Pos) = \frac{0 + 1}{13} = \frac{1}{13}$$



Paso 5: Laplace Smoothing

Fórmula general:
$$P(w|c) = \frac{\text{count}(w, c) + 1}{N_c + |V|}$$

Probabilidades en Negativo

$$N_{neg} + |V| = 4 + 7 = 11$$

Ejemplos:

$$P(mala|Neg) = \frac{1 + 1}{11} = \frac{2}{11}$$

$$P(película|Neg) = \frac{2 + 1}{11} = \frac{3}{11}$$

$$P(excelente|Neg) = \frac{0 + 1}{11} = \frac{1}{11}$$



Paso 6: Clasificación de una nueva frase

Fórmula de Naïve Bayes: $P(c|d) \propto P(c) \prod_i P(w_i|c)$

Nueva frase:

$d = \text{"excelente película"}$

Para Positivo:

$$P(Pos|d) \propto P(Pos) \cdot P(excelente|Pos) \cdot P(película|Pos)$$

$$= 0.5 \cdot \frac{2}{13} \cdot \frac{3}{13} = \frac{3}{169}$$

Modelo de clasificación, Naïve Bayes



Paso 6: Clasificación de una nueva frase

Fórmula de Naïve Bayes: $P(c|d) \propto P(c) \prod_i P(w_i|c)$

Nueva frase:

$d = \text{"excelente película"}$

Para Negativo:

$$P(Neg|d) \propto P(Neg) \cdot P(excelente|Neg) \cdot P(película|Neg)$$

$$= 0.5 \cdot \frac{1}{11} \cdot \frac{3}{11} = \frac{3}{242}$$

Modelo de clasificación, Naïve Bayes



Paso 6: Clasificación de una nueva frase

Fórmula de Naïve Bayes: $P(c|d) \propto P(c) \prod_i P(w_i|c)$

Nueva frase:

$d = \text{"excelente película"}$

Positivo

Decisión final:

$$\frac{3}{169} > \frac{3}{242}$$

Practiquemos lo visto hasta ahora



Link al Colab



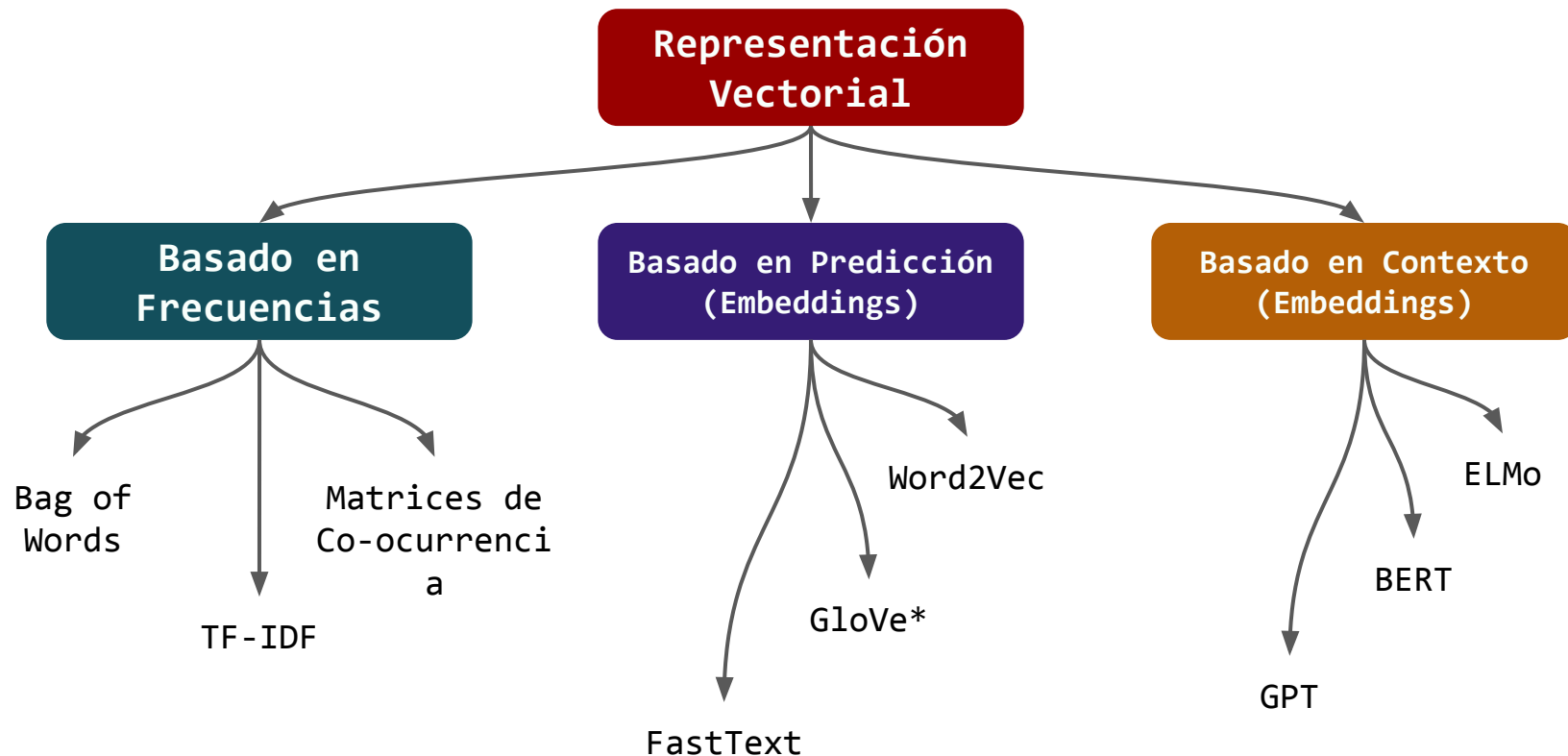
LINK



Link al Colab



LINK



NLP Clásico vs NLP + DL vs LLMs



NLP Clásico

- 1.- Texto
- 2.- Tokenización
- 3.- Stop words removal
- 4.- Stemming / Lemmatization
- 5.- Vectorización (BoW, TF-IDF)
- 6.- Modelo ML

NLP + DL

- 1.- Texto
- 2.- Tokenización
- 3.- Subword tokenization (BPE, WordPiece)
- 4.- Embeddings
- 5.- Modelo

LLMs

- 1.- Texto
- 2.- Tokenizer (BPE / WordPiece / SentencePiece)
- 3.- IDs de tokens
- 4.- Transformer

Sobre el uso de LLMs y asistentes de código en la materia...



¡¡Totalmente permitidos!!

Especificar modelo/asistente usado, fecha y prompts utilizados.

Recuerden que los LLM son asistentes para su trabajo y no un reemplazo de su trabajo.